

Reviewer Pack — Minimal Brauer Group Order Correlation with Frege Proof Size

Entry 0a5352a52fb4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 05:19:50 UTC

Minimal Brauer Group Order Correlation with Frege Proof Size

Entry ID: 0a5352a52fb4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 05:19:50 UTC

1. Conjecture

Field A (mathematical branch): Brauer Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

For every CNF φ with m clauses and n variables, the order of the minimal Brauer group associated with the algebraic structure of φ is linearly correlated with its Frege proof size $f(\varphi)$, such that $\text{Order}(\text{BrauerGroup}(\varphi)) = \Theta(f(\varphi))$.

Rationale (proposer's reasoning):

Brauer theory provides tools to study the structure of algebras and their representations, which could potentially reveal underlying symmetries or hidden complexity in boolean functions. By linking this with Frege proof size, we may uncover new connections between algebraic structures and computational complexity.

Taxonomy category: brauer_theory_frege_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): efd4554c0fb98383

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if the Pearson correlation coefficient (r) between the minimal Brauer group order and the Frege proof size for all CNFs is ≥ 0.7 , with a p -value ≤ 0.01 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Minimal Brauer Group order" AND "Frege proof complexity" - "algebraic structure" AND CNF AND "Brauer group order" - "correlation" AND "Frege proof size" AND "minimal Brauer group"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random CNF with m clauses and n variables
    n = random.randint(5, 10)
    m = random.randint(5, 40)
    cnf = []
    for _ in range(m):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        cnf.append(clause)

    # Constructive mapping from CNF to algebraic structure
    # This is a placeholder function. Replace with actual implementation.
    def construct_algebraic_structure(cnf):
        # Placeholder return value
        return random.randint(1, 100)

    algebraic_structure = construct_algebraic_structure(cnf)

    # Calculate the minimal Brauer group order (placeholder)
    def min_brauer_group_order(algebraic_structure):
        # Placeholder return value
        return algebraic_structure

    brauer_group_order = min_brauer_group_order(algebraic_structure)

    # Compute Frege proof size (placeholder)
    def frege_proof_size(cnf):
        # Placeholder return value
        return random.randint(1, 100)

    proof_size = frege_proof_size(cnf)
```

```

# Store the results
result = {
    "metric_name": "Brauer Group Order vs Frege Proof Size",
    "metric_value": brauer_group_order,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

return result

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

# Compute mean/std of metric_value and fraction of seeds where conjecture_holds
if not results:
    print("RESULT: INCONCLUSIVE no_results")
    exit(0)

total_metric = sum(result["metric_value"] for result in results)
mean_metric = total_metric / len(results)

variance = sum((result["metric_value"] - mean_metric) ** 2 for result in results) / len(results)
std_metric = math.sqrt(variance)

support_count = sum(1 for result in results if result["conjecture_holds"])
support_fraction = support_count / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    counterexample = next(result for result in results if not result["conjecture_holds"])["counterexample"]
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
'conjecture_holds': False, 'counterexample': 'mapping_undefined'
```

```

TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 25, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 56, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 46, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 13, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 95, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 59, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 6, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 26, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 7, 'instances_tested': 10}
TRIAL: {'metric_name': 'Brauer Group Order vs Frege Proof Size', 'metric_value': 19, 'instances_tested': 10}
RESULT: FALSIFIED counterexample="mapping_undefined" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder functions for constructing the algebraic structure, calculating the minimal Brauer group order, and computing the Frege proof size. These placeholders do not implement the actual mathematical operations required to verify the conjecture, making it impossible to confirm or refute the statement with this test.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test code uses placeholder functions for constructing the algebraic structure, calculating the minimal Brauer group order, and computing the Frege | next: Develop a proper implementation of the necessary mathematical operations to construct the algebraic structures, calculate the minimal Brauer group orders, and compute the Frege proof sizes for CNFs, then retest the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13232
2	preregistration	ollama_remote	glm4:latest	0	0	9113
3	novelty	ollama_remote	glm4:latest	0	0	8346
4	novelty	ollama_remote	glm4:latest	0	0	8907
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15334
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25589
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11680
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9828
9	critic	ollama_remote	glm4:latest	0	0	14141
10	judge	ollama_remote	glm4:latest	0	0	13244

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129413 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/0a5352a52fb4.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0a5352a52fb4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0a5352a52fb4.tar.gz` (if generated)